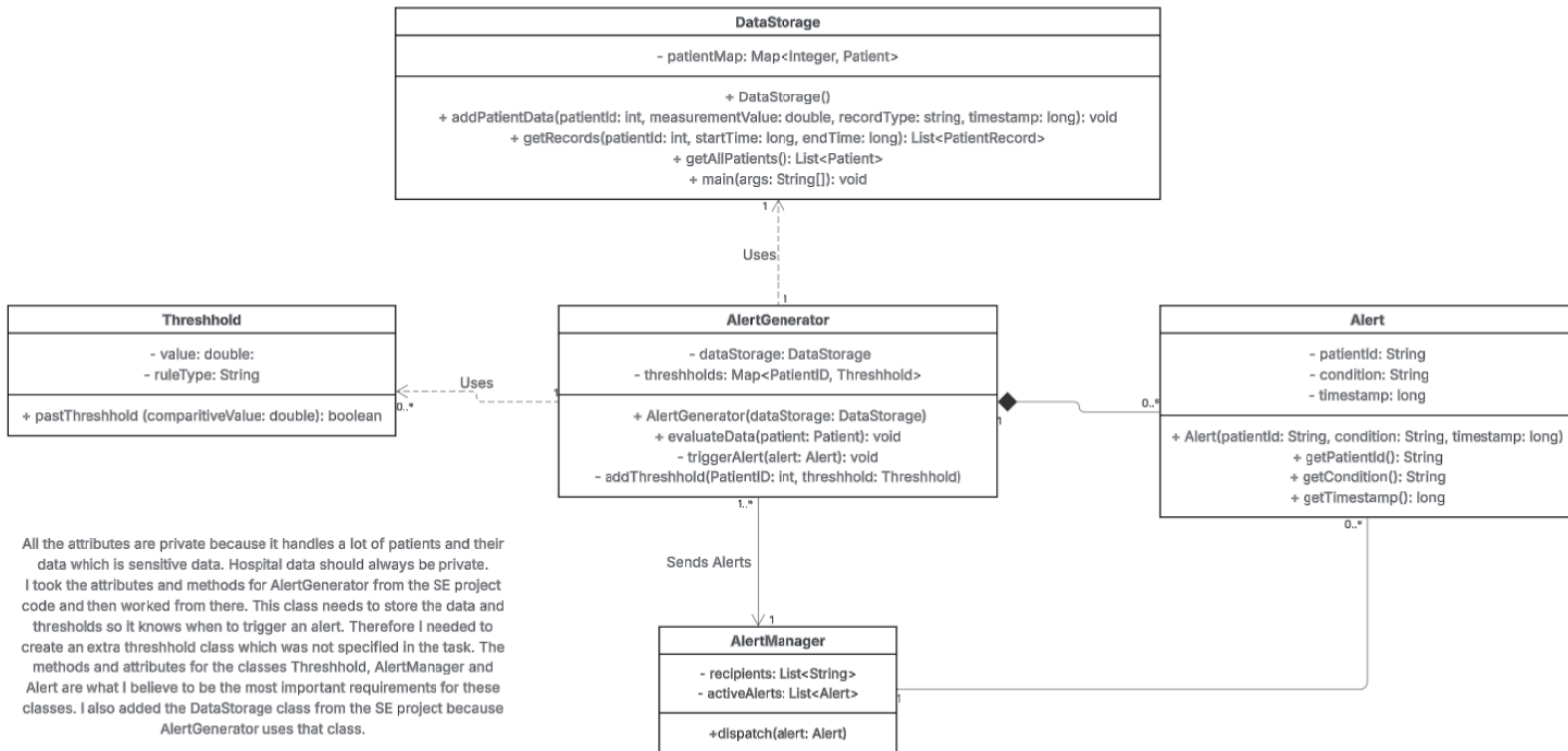
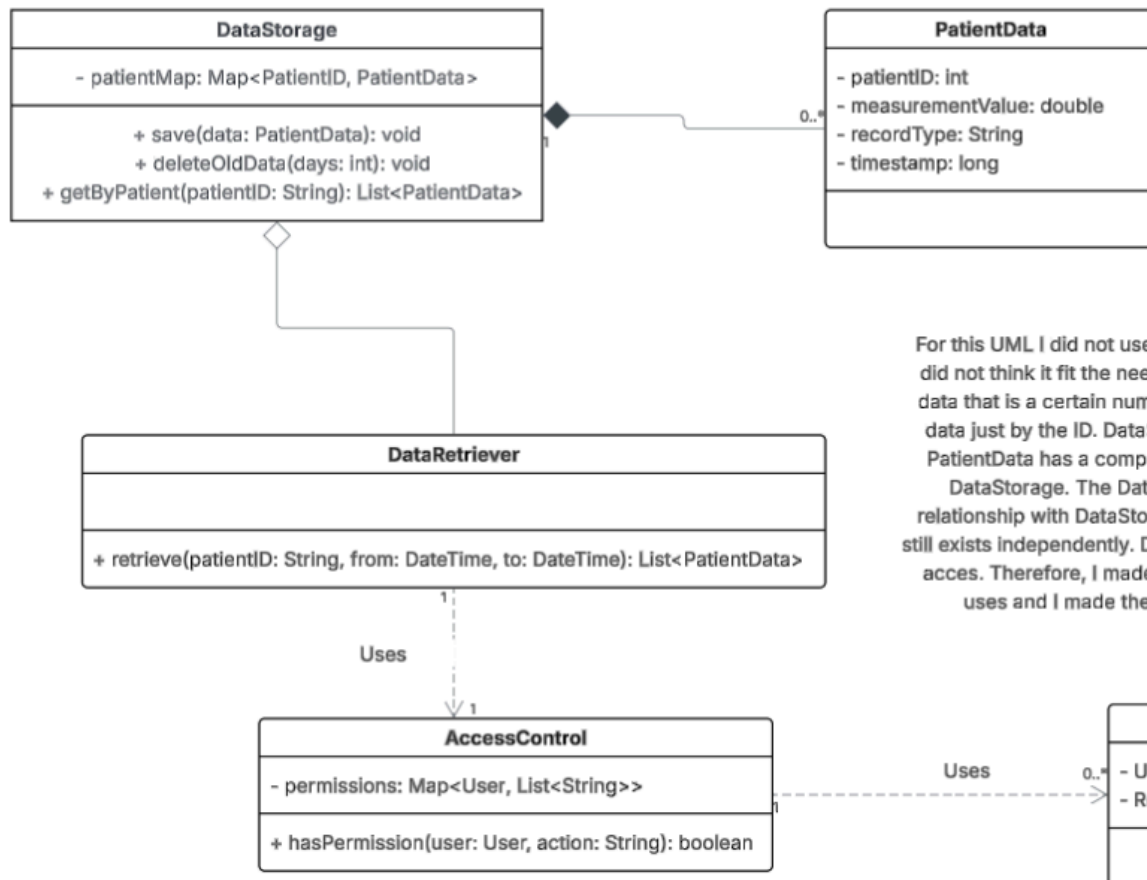


1) Alert Generation System



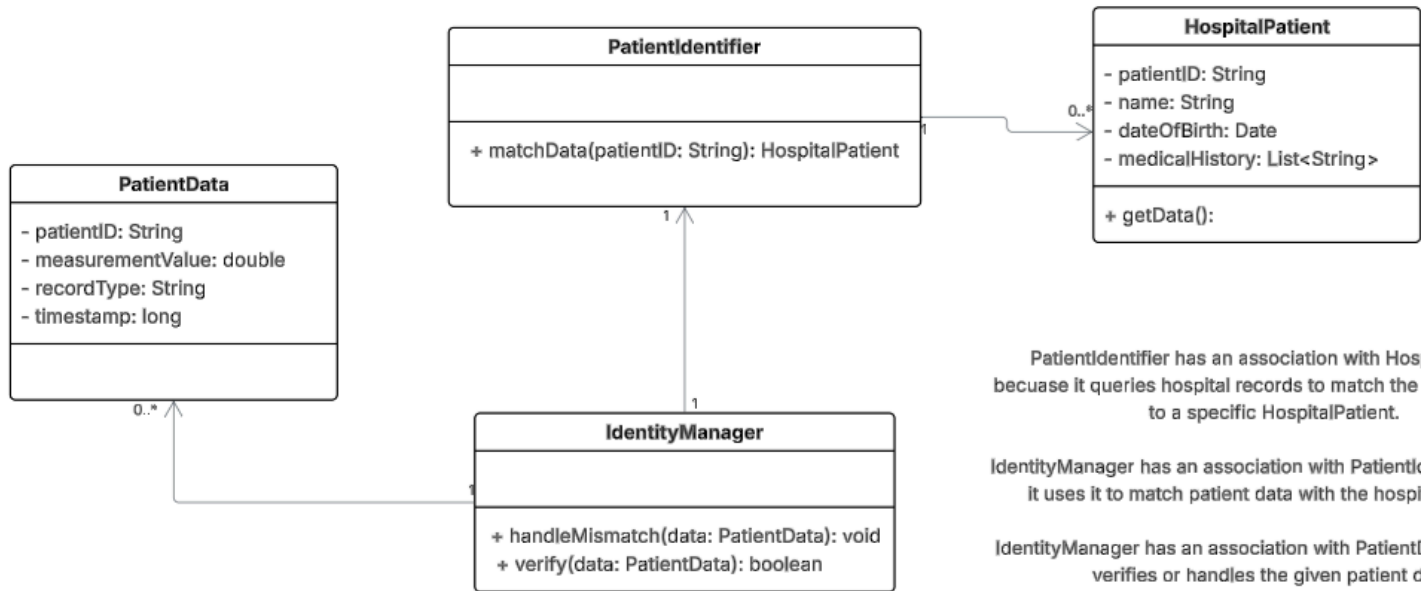
All the attributes are private because it handles a lot of patients and their data which is sensitive data. Hospital data should always be private. I took the attributes and methods for AlertGenerator from the SE project code and then worked from there. This class needs to store the data and thresholds so it knows when to trigger an alert. Therefore I needed to create an extra threshold class which was not specified in the task. The methods and attributes for the classes Threshold, AlertManager and Alert are what I believe to be the most important requirements for these classes. I also added the DataStorage class from the SE project because AlertGenerator uses that class.

2) Data Storage System



For this UML I did not use the **DataStorage** class from the SE project as I did not think it fit the needs of this task. Instead it can save data, delete data that is a certain number of days old and can get a specific patients data just by the ID. **DataStorage** can store all **PatientData** which is why **PatientData** has a composition relationship - it would not exist without **DataStorage**. The **DataRetriever** class I believe has an aggregation relationship with **DataStorage** as if **DataStorage** doesn't exist then it can still exist independently. **DataRetriever** also needs to be able to control the access. Therefore, I made the **AccessControl** class which **DataRetriever** uses and I made the **User** class which **AccessControl** will use.

3) Patient Identification System

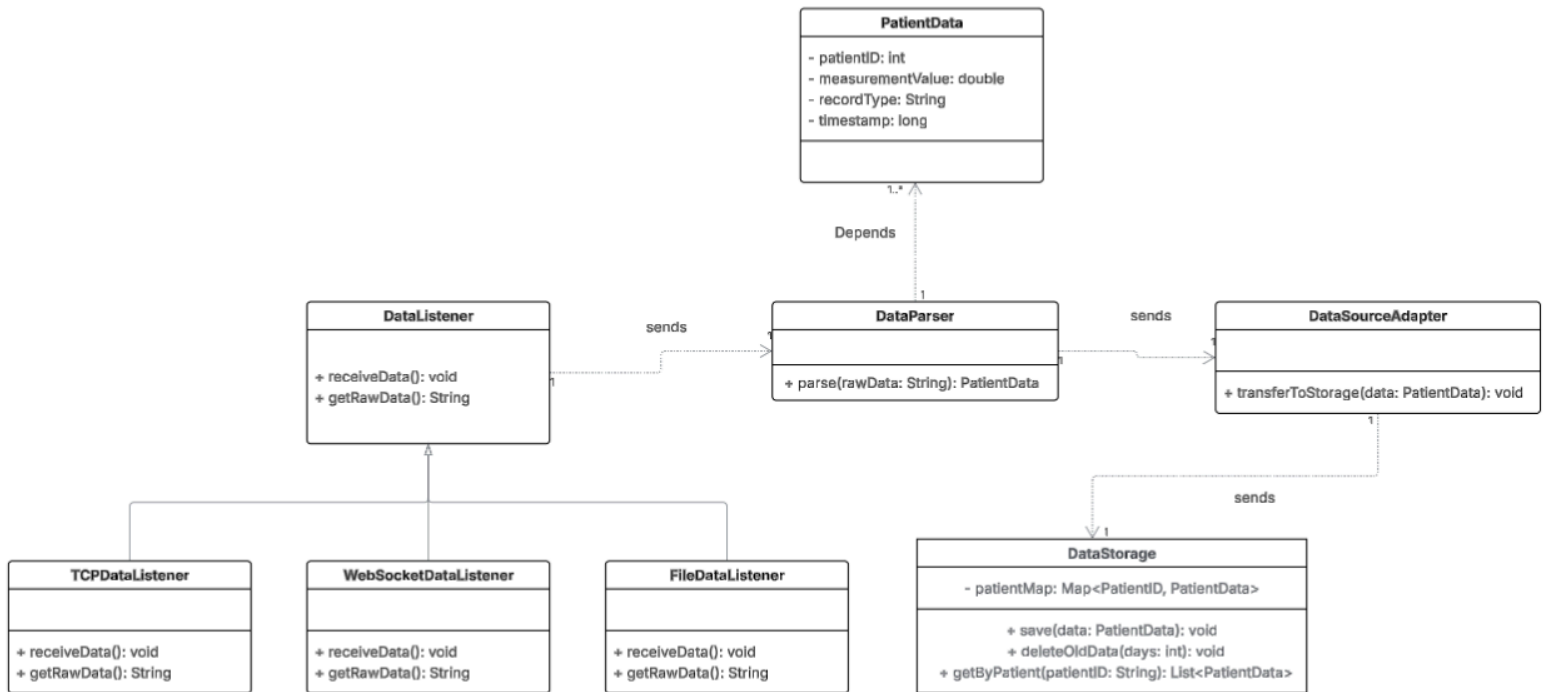


PatientIdentifier has an association with HospitalPatient because it queries hospital records to match the given patientID to a specific HospitalPatient.

IdentityManager has an association with PatientIdentifier because it uses it to match patient data with the hospital records.

IdentityManager has an association with PatientData because it verifies or handles the given patient data.

4) Data Access Layer



The Data Listener class is an interface to three subclasses which is why they share the same methods. DataListener then sends the raw data to the DataParser. DataParser returns PatientData which is why it depends on the PatientData class. DataParser then sends the parsed data to the DataSourceAdapter which uses the transferToStorage method to send the data to the DataStorage class. I gave all these classes a dependency arrow / dependency usage arrow because they can exist without the other classes.

Note: I have used patient data as an example, but this system would work with any data which is why I did not use a composition relationship between DataParser and PatientData.